Lab Manual # 09

Structured Query Language (SQL)

What is T-SQL?

Transact-SQL (T-SQL) is an extension of SQL (Structured Query Language) developed by Microsoft and Sybase. It is primarily used in Microsoft SQL Server and Sybase Adaptive Server. T-SQL enhances standard SQL with additional features that allow for more complex data manipulation and retrieval, making it a powerful tool for developers and database administrators.

Key Features of T-SQL:

1. **Procedural Programming:**
    - T-SQL incorporates procedural programming constructs, enabling users to define procedures, loops, and conditions within SQL statements. This allows for more dynamic and complex database operations.
2. **Variables and Data Types:**
    - T-SQL supports the declaration of variables, allowing users to store data temporarily for processing. It also provides various data types, including numeric, string, date/time, and binary types.
3. **Control-of-Flow Language:**
    - T-SQL includes control-of-flow statements, such as IF...ELSE, WHILE, GOTO, and BEGIN...END, allowing for conditional execution and looping, which can enhance the logic of SQL scripts.
4. **Error Handling:**
    - T-SQL offers robust error handling capabilities through TRY...CATCH blocks, enabling developers to manage exceptions effectively and maintain the stability of their applications.
5. **Stored Procedures and Functions:**
    - T-SQL allows for the creation of stored procedures and user-defined functions, encapsulating business logic and promoting code reuse. These can be called within other T-SQL statements, providing modularity in database applications.
6. **Triggers:**
    - T-SQL supports triggers, which are special types of stored procedures that automatically execute in response to certain events on a table, such as INSERT, UPDATE, or DELETE. This enables automatic auditing, validation, or cascading actions.
7. **Cursors:**

- T-SQL provides cursors for row-by-row processing of query results, allowing for complex operations that cannot be accomplished with standard set-based SQL operations.

8. **Rich Built-in Functions:**
   - T-SQL includes a variety of built-in functions for string manipulation, date/time operations, mathematical calculations, and aggregate functions, enhancing its capability for data processing and analysis.
9. **Common Table Expressions (CTEs):**
   - T-SQL supports CTEs, which provide a way to write recursive queries and improve the readability of complex queries.
10. **Integration with SQL Server Features:**
    - T-SQL is deeply integrated with SQL Server features, such as transactions, locking mechanisms, and indexing, allowing for optimized performance and data integrity.

Use Cases:

- **Data Analysis:** T-SQL can be used to perform complex queries, aggregations, and reporting.
- **Application Development:** Developers can use T-SQL to build applications that require database interactions.
- **Database Management:** Database administrators utilize T-SQL for managing databases, backups, and performance tuning.

Today's Lab Concepts in T-SQL

1. **Variables in T-SQL**
2. **Constants and literals in T-SQL**
3. **Print/Display in T-SQL**
4. **If-Else in T-SQL**
5. **Switch (CASE) in T-SQL**
6. **While loop in T-SQL**
7. **For loop (simulated) in T-SQL**
8. **Exit loop in T-SQL**
9. **Loop label in T-SQL**
10. **Goto in T-SQL**
11. **Stored procedure in T-SQL**
12. **Function in T-SQL**
13. **Cursor in T-SQL**
14. **Exception handling in T-SQL**
15. **Triggers in T-SQL**
16. **Package (simulated as modules) in T-SQL**

1. Variables in T-SQL

Explanation:

Variables in T-SQL are used to store temporary data. They are declared using the DECLARE statement, and values can be assigned using the SET or SELECT statements. Variables allow you to store and manipulate data during the execution of your queries or procedures.

Example:

```
DECLARE @employee_name VARCHAR(50);   -- Declaring a variable
SET @employee_name = 'John Doe';      -- Assigning a value
PRINT 'Employee Name: ' + @employee_name; -- Outputting the value
```

```
DECLARE @name VARCHAR(50);
DECLARE @salary INT;

SELECT @name = emp_name, @salary = salary
FROM Employees
WHERE empid = 4;

PRINT 'Employee Name: ' + @name;
PRINT 'Employee Salary: ' + CAST(@salary AS VARCHAR);
```

Key Points:

- You must declare the variable before using it.
- Variable names begin with @.
- Data Types for Variables: `INT`, `VARCHAR`, `DECIMAL`, `DATE`, `CHAR`, `BIT`, etc.

---

2. Constants and Literals in T-SQL

Explanation:

T-SQL does not have a dedicated keyword for **constants** like PL/SQL. However, you can treat a variable as a constant by simply not re-assigning it after the initial value is set. **Literals** in T-SQL are fixed values that are used directly in the code. They are not stored in variables and are not the result of any expressions. T-SQL supports several types of literals, including numeric, string, and date literals.

Example:

```
DECLARE @PI DECIMAL(10,4) = 3.1416; -- Treat this as a constant
PRINT 'Value of Pi: ' + CAST(@PI AS VARCHAR);
```

```
SELECT 100;          -- Integer literal
SELECT -25;          -- Negative integer literal
SELECT 3.14159;      -- Decimal literal
SELECT 'Hello, World'; -- String literal
```

Key Points:

- You can assign values during declaration.

- Reassigning the value would mean it’s no longer treated as a constant.

---

3. Print/Display in T-SQL

Explanation:

This is the simplest possible script that outputs a string. In T-SQL, the PRINT statement is used to display text in the output window.

Example:

```
PRINT 'Hello, World!';
```

Key Points:

- PRINT is used to output any message.
- It's the most basic way to interact with users through output.

---

4. If-Else in T-SQL

Explanation:

Conditional logic is implemented using the IF...ELSE statement. This allows you to execute different blocks of code depending on whether a condition is true or false.

Example:

```
DECLARE @age INT = 21;
IF @age >= 18
BEGIN
    PRINT 'You are an adult.';
END
ELSE
BEGIN
    PRINT 'You are not an adult.';
END
```

Key Points:

- IF checks a condition.
- ELSE executes if the IF condition is false.

---

5. Switch (CASE) in T-SQL

Explanation:

T-SQL uses the CASE expression to handle multiple conditions, similar to a switch-case structure in other languages. This is useful when you have several possible values for a variable and want different actions based on that value.

Example:

```
DECLARE @grade CHAR(1) = 'B';
SELECT
    CASE @grade
        WHEN 'A' THEN 'Excellent'
        WHEN 'B' THEN 'Good'
        ELSE 'Needs Improvement'
    END AS Performance;
```

```
UPDATE Employees
SET bonus =
    CASE
        WHEN salary >= 70000 THEN 5000
        WHEN salary >= 60000 THEN 3000
        ELSE 1000
    END;
```

```
SELECT emp_name, salary
FROM Employees
ORDER BY
    CASE
        WHEN salary >= 70000 THEN 1
        WHEN salary >= 60000 THEN 2
        ELSE 3
    END,
    SAL DESC;
```

```
SELECT
    SUM(CASE WHEN department = 'Sales' THEN salary ELSE 0 END) AS SalesTotalSalary,
    SUM(CASE WHEN department = 'IT' THEN salary ELSE 0 END) AS ITTotalSalary
FROM Employees;
```

Key Points:

- The CASE expression returns a result based on multiple conditions.
- It is often used within SELECT queries.

---

6. While Loop in T-SQL

Explanation:

The WHILE loop in T-SQL repeats a block of code as long as a given condition is true. It's commonly used for iterative processes.

Example:

```
DECLARE @count INT = 1;
```

```
WHILE @count <= 5
BEGIN
    PRINT 'Count: ' + CAST(@count AS VARCHAR);
    SET @count = @count + 1;
END;
```

Key Points:

- The condition is checked at the beginning of each iteration.
- The loop continues until the condition is false.

---

7. For Loop in T-SQL

Explanation:

Unlike PL/SQL, T-SQL does not have a direct FOR loop. However, you can simulate a FOR loop using a WHILE loop. The loop runs a specific number of times by controlling the increment of a variable.

Example:

```
DECLARE @i INT = 1;
WHILE @i <= 10
BEGIN
    PRINT 'Iteration: ' + CAST(@i AS VARCHAR);
    SET @i = @i + 1;
END;
```

Key Points:

- Use WHILE to mimic FOR behavior.
- Ensure to increment or decrement the loop counter to avoid infinite loops.

---

8. Exit Loop in T-SQL

Explanation:

In T-SQL, you can use loops to perform iterative tasks within a batch, stored procedure, or function. However, sometimes you may need to exit the loop before it completes all iterations. To achieve this, you can use the **BREAK** and **CONTINUE** control-of-flow statements.

BREAK Statement

- **BREAK**: Exits the innermost loop immediately and resumes execution after the loop.
- **Example**:

```
DECLARE @counter INT = 1;
WHILE @counter <= 5
BEGIN
    PRINT 'Counter: ' + CAST(@counter AS VARCHAR);
    SET @counter = @counter + 1;
    IF @counter > 3
        BREAK; --Exit the loop
```

```
    END;
```

Key Points:

- Use BREAK to exit a loop prematurely.
- Loops in T-SQL include WHILE.

`CONTINUE` Statement

- **`CONTINUE`**: Skips the remaining statements in the current iteration and starts the next iteration of the loop. This is useful when certain conditions need to be skipped without breaking the entire loop.

Example:

```
DECLARE @i INT = 1;
WHILE @i <= 10
BEGIN
    IF @i % 2 = 0
    BEGIN
        SET @i = @i + 1;
        CONTINUE; -- Skip even numbers
    END;
    PRINT 'Odd Number: ' + CAST(@i AS VARCHAR);
    SET @i = @i + 1;
END;
```

Key Points:

- CONTINUE skips the remaining statements in the current iteration.
- The loop moves on to the next iteration after CONTINUE.

---

9. Loop Label in T-SQL

Explanation:

While T-SQL does not use labeled loops like PL/SQL, you can structure complex queries to logically organize control flow. However, GOTO (explained below) is used for jumping to specific sections of code.

Example:

```
DECLARE @i INT = 1;
LOOP_LABEL:
WHILE @i <= 5
BEGIN
    PRINT 'Number: ' + CAST(@i AS VARCHAR);
    SET @i = @i + 1;
END;

-- You can "jump" to LOOP_LABEL using GOTO
```

Key Points:

- No direct labels for loops.

- Labels can be mimicked using GOTO.

---

10. Goto in T-SQL

Explanation

The GOTO statement allows you to jump to a specific labeled section in the code. This is similar to the GOTO in PL/SQL. It is generally used for error handling or exiting deeply nested loops.

Example:

```
DECLARE @i INT = 1;

LOOP_LABEL:
WHILE @i <= 5
BEGIN
    PRINT 'Number: ' + CAST(@i AS VARCHAR);
    SET @i = @i + 1;

    -- If @i equals 3, jump to another section of the code using GOTO
    IF @i = 3
    BEGIN
        GOTO exit_loop;  -- Simulating an exit from the loop via GOTO
    END
END;

-- Define a labeled section to exit the loop
exit_loop:
PRINT 'Exited the loop at i = ' + CAST(@i AS VARCHAR);
```

Key Points:

- GOTO transfers control to a label within the code.
- Use it sparingly as it can make code hard to follow.